

EECE 430L | Lab #8: Mobile I

Introduction

This lab will develop a native Android application for the currency exchange project. It will produce an Android mobile interface capable of functionality similar to the frontend web, namely viewing the current exchange rates, and registering a new user transaction. This application will integrate with our developed backend.

We will use Android Studio and the built-in emulator for development and testing.

However if you possess an Android device, you can also use it to test the application (with some minor setup).

Make sure to refer to the Android Developer Documentation throughout your work to look up and gain a basic understanding of new constructs as you come across them within the lab.

When not specified, add the appropriate imports alongside the described code blocks. (Android Studio will typically provide you with recommendations for doing so).

Phase 1. Exchange Rate UI

1. As a first step, accept the GitHub assignment invitation to create a new repository for the mobile project, and clone it on your machine.
2. Open Android Studio, and create a new project using the 'Empty Activity' template. When prompted, fill the different fields accordingly:
 - Name: Currency Exchange
 - Package name: `com.PERSONAL_IDENTIFIER_OR_USERNAME.currencyexchange`
 - Save location: Location of the repository
 - Language: Kotlin
 - Minimum SDK: API 21
 - Use legacy android.support libraries: No
3. Once the project is created, take a few moments to explore the project, and the Android Studio IDE. Take a look at the different files and the folder structure. Note that at the top level is your project, and within it is a single `app` module that contains your application. Generally speaking a single project can house several related modules. Within the app module, the `src` directory will contain all the effective content of the application.
4. Run the application within your emulator. If the application is running in the emulator without any issues, you can create a commit with this basic setup. There may be some build issues that take place which will require you to make appropriate changes to your environment.
5. We will begin by setting up the UI elements needed to display the exchange rates. In Android development, component UIs are generally defined in XML files. An application's content is generally split between the code in the `java` directory, and the resources which include layouts and images/icons in the `res` directory. Find and open the `activity_main.xml` file, which contains the layout for the single page of our application. You can see the contents of this layout currently rendered in the emulator. Note that Android Studio provides you with the ability to graphically modify

the UI of the application, in addition to directly modifying the XML. Select the 'Split' view in AS to allow you to edit the XML, but also see the changes directly in the design pane.

6. As a first step, delete the unneeded 'Hello World' TextView. This will leave us with only a ConstraintLayout. This Layout is a type of *ViewGroup* that allows us to define the positioning of its children elements using certain *constraints* that we will see in more detail.
7. Add the following element within the ConstraintLayout, to serve as a sort of page header. As you can see, in XML layouts, elements can be defined and each given a set of attributes to configure them. The following is worth noting about the attributes of this component:
 - `textStyle`, `text`, and `textSize` should be fairly self explanatory.
 - `layout_width` and `layout_height` specify the size that this element should take up, and in this case both of these values are set to `wrap_content`. This value means that the view will take as much space as its contents require. `layout_marginTop` specifies a certain amount of spacing from the top of the element to the next nearest element.
 - `layout_constraintLeft_toLeftOf`, `layout_constraintRight_toRightOf`, and `layout_constraintTop_toTopOf` are attributes that are used to define how an element should be positioned specifically when it is inside of a ConstraintLayout. Refer to this [documentation](#) to understand the basics of how these specific attributes are to be used.

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:textStyle="bold"
    android:text="LBP Exchange Rate"
    android:textSize="24sp"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

8. You should be able to see the new header element in the design pane in AS. You can also re-run the application to see the changes within the emulator.
9. We will display the two exchange rates side by side within the page, and each exchange rate will be represented by a label specifying if it is the 'Buy USD' or 'Sell USD' rate, and beneath the actual rate will be shown. We will achieve this UI by placing two ViewGroups side by side, and within each one will be the label and value for each exchange rate. Add the following two containers within your ConstraintLayout. Take some time to understand the different attributes being used and look up the elements/attributes you are unfamiliar with.

```
<LinearLayout
    android:id="@+id/vwBuyUsd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHorizontal_weight="1"
    app:layout_constraintRight_toLeftOf="@id/vwSellUsd"
```

```

        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.3">

</LinearLayout>

<LinearLayout
    android:id="@+id/vwSellUsd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintHorizontal_weight="1"
    app:layout_constraintLeft_toRightOf="@id/vwBuyUsd"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.3">

</LinearLayout>

```

10. Since the above `LinearLayout` elements are *ViewGroups* and not actual *Views*, do not expect anything to show in your design yet. Add a `TextView` element within each `LinearLayout` in order to serve as a label for each exchange rate. You can use the existing header `TextView` for guidance, but note that here the `layout_constraint` attributes won't be needed. Also make sure that the `fontSize` of these labels are smaller than the page header, for UI purposes.
11. Add another `TextView` under each of the `TextViews` we just created, where each new `textview` will hold the corresponding exchange rate. Give these `TextViews` the following ID values: `"@+id/txtBuyUsdRate"`, and `"@+id/txtSellUsdRate"` appropriately. Also set their `text` attributes with appropriate placeholders.
12. We have now completed the UI setup.

Phase 2. Fetching and Displaying Exchange Rates

13. Find and open the `MainActivity.kt` file. This file defines '`MainActivity`' which is an Android activity, and which in our case is the single activity of the application. Note the presence of the `onCreate` function, which overrides this *lifecycle* method of the activity to define what our activity should do when it is first created. In our case, the only operation being executed is `setContentView(R.layout.activity_main)`, which is what indicates that the layout content for this Activity is the `activity_main.xml` layout file that we've been working on.
14. We will prepare to gain access to the exchange rate `TextViews` so that when we have fetched the exchange rates, we can update the appropriate `TextViews` to display them. Do so by first defining the following variables at the beginning of the Activity in order to maintain references to the `TextViews`. Add the appropriate needed import statements.

```
private var buyUsdTextView: TextView? = null
private var sellUsdTextView: TextView? = null
```

15. Now within the `onCreate` function, after the `setContentView` statement, we will obtain reference to the TextViews and store them in the above created variables by adding the following code:

```
buyUsdTextView = findViewById(R.id.txtBuyUsdRate)
sellUsdTextView = findViewById(R.id.txtSellUsdRate)
```

16. Now that the above is ready, we will now setup the needed code to allow us to communicate to our backend in order to fetch the exchange rates. We will do so with the help of a library called Retrofit, which makes HTTP (specifically using JSON) communication easier. At this stage, make sure that your backend server is running locally.
17. Firstly we must declare that our application requires internet access, which on Android is a system permission that must be specified. To do so, open the `AndroidManifest.xml` file. This file contains various configuration information about our project. Add the following element before the `application` tag.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

18. Android, particularly in newer SDK versions, requires HTTP communication to take place over SSL for security purposes. For our development, since we are integrating with our local backend server that does not have SSL enabled, we will have to specifically enable allowing non-SSL communication. We can do so by adding the following attribute within the `application` tag, also inside our `AndroidManifest.xml` file.

```
android:usesCleartextTraffic="true"
```

19. Now to specify that our project will use the Retrofit library, open the application's gradle file. Note that there are two gradle files, one for the entire project, and one only for the application module. Open the one corresponding to the application module. Within the dependencies block of this file, add the following lines. This will specify which libraries we will use, and their versions.

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0'
implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
```

20. Now setup the following file structure to handle server communication. Within the package that contains the `MainActivity.kt` file, create a new package named `api`. Within the `api` package create a Kotlin file named `retrofit.kt`. Also within the `api` package, create a package named `model` that will store our data models.

21. Within the `model` package add a Kotlin file that will contain a class. Name this file `ExchangeRates.kt`. This file/class will hold a class that will represent the response received from our backend when fetching the exchange rates. Within the `ExchangeRates.kt` file, replace the default created class with the following content. Here we declare the fields we expect to receive from the backend. We declare using the `SerializedName` annotation the name of the variables as they will be received from the backend, as compared to the variable name to be used within our class.

```
import com.google.gson.annotations.SerializedName

class ExchangeRates {
    @SerializedName("usd_to_lbp")
    var usdToLbp: Float? = null

    @SerializedName("lbp_to_usd")
    var lbpToUsd: Float? = null
}
```

22. Now within the `retrofit.kt` file, add the following content. Import additionally the `ExchangeRates` class we have just created (AS will probably provide you with a hint to do so). Very briefly, this will allow other code within our project to easily make requests to the backend endpoints declared in this file, as we will see in following steps. Note that the `interface Exchange` section is the primary section where the parameters of the backend that we will be communicating with are set. Feel free to review the details of this file to gain a familiarity with Kotlin and the Retrofit library. Note that the IP `10.0.2.2` is used for the emulator to communicate with localhost on the machine that is running the emulator. If you plan to test on a physical device, make sure the device is connected to the same local network as your machine, and use the IP address of your machine.

```
import retrofit2.Call
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET

object ExchangeService {
    private const val API_URL: String = "http://10.0.2.2:5000"

    fun exchangeApi(): Exchange {
        val retrofit: Retrofit = Retrofit.Builder()
            .baseUrl(API_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        // Create an instance of our Exchange API interface.
        return retrofit.create(Exchange::class.java)
    }
}

interface Exchange {
    @GET("/exchangeRate")
    fun getExchangeRates(): Call<ExchangeRates>
}
```

```
}  
}
```

23. We will now return to the `MainActivity` to make use of this infrastructure that will allow us to easily communicate with our backend endpoints. Add the following import statements, in addition to importing the `ExchangeRate` and `ExchangeService` objects that we have created. Also, add to the end of the `onCreate` function, the following code. This will use the `ExchangeService.exchangeApi` function to get a reference to the Retrofit object that represents our backend. We will then make an HTTP GET request to the `exchangeRate` endpoint using the `getExchangeRates` function, and we will specify in a callback what to do when the request completes (successfully or not).

```
import retrofit2.Call  
import retrofit2.Callback  
import retrofit2.Response
```

```
ExchangeService.exchangeApi().getExchangeRates().enqueue(object :  
    Callback<ExchangeRates> {  
        override fun onResponse(call: Call<ExchangeRates>, response:  
            Response<ExchangeRates>) {  
            val responseBody: ExchangeRates? = response.body();  
            ...  
        }  
  
        override fun onFailure(call: Call<ExchangeRates>, t: Throwable)  
        { return;  
          TODO("Not yet implemented")  
        }  
    })
```

24. In place of the missing block of code, add the necessary code to extract from the `responseBody` object the exchange rates, and then use them to update the two `TextViews` in our UI. Take a moment to figure out and research the needed Kotlin constructs and Android methods to do so.
25. After completing the above, you should be able to re-run the application and see the present exchange rates within the app. If you are facing issues, explore how to add log statements to your code and how to view logs within AS while the app is running, to help you debug. Once completed and tested, commit this phase of changes.

Phase 3. Adding New Transactions

26. We will add into the application the ability for the user to add new transactions that they have made, for them to be registered within our backend. We will make use of elements from the Material UI library to do so. This dependency is already included by default in our Gradle file.
27. We will add a [Floating Action Button](#) for users to add new transactions. As a first step, add the following to the `activity_main.xml` file, at the top level within the `ConstraintLayout`.

```

<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    android:layout_margin="32dp"
    android:src="@drawable/ic_baseline_add_24"
    android:contentDescription="Add Transaction"
/>

```

28. This button references an icon that doesn't exist yet. To create the '+' icon that we will use within the FAB button, do the following. First, right-click in AS on the 'res' directory, and select 'new' -> 'vector asset'. Within the 'Clip Art' field, select the icon named 'add'. The name should be set automatically to the name that is referenced from the FAB above. Set the color to white, and create the new icon. Icons and assets of this nature are stored within the `drawable` directory.
29. Open the themes file under `res/values/themes.xml`, and change the value of the `colorOnSecondary` entry to white. You can also change the value of the `colorSecondary` entry to change the color of the FAB. You can use `@color/purple_500` as a value unless you would like to set another color.
30. We will now create a `Dialog` element that will pop-up when the user presses the FAB and will allow the user to enter the transaction details. Do so by first creating a new layout within the layout directory named `dialog_transaction.xml` with a `ConstraintLayout` as its root.
31. Our dialog will contain two inputs to capture the USD and LBP values of the transaction, in addition to a radio button group to select the type of the transaction. Add the following to create the first input. This will create a `TextInput` from the Material library, which will provide us with some styling and functionality out of the box. This contains a `TextInputLayout` element, and inside of it the `TextInputEditText` that actually captures the user input.

```

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/txtInptUsdAmount"

style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="USD Amount"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent">

    <com.google.android.material.textfield.TextInputEditText
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:inputType="number"
        android:singleLine="true" />

</com.google.android.material.textfield.TextInputLayout>

```

32. Following the above element, add a new similar element to capture the LBP amount, setting its id attribute to `"@+id/txtInptLbpAmount"` and making the appropriate adjustments.
33. Finally, add the following to create the radio button group.

```
<RadioGroup
    android:id="@+id/rdGrpTransactionType"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@id/txtInptLbpAmount">
    <RadioButton
        android:id="@+id/rdBtnBuyUsd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Buy USD" />

    <RadioButton
        android:id="@+id/rdBtnSellUsd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Sell USD" />
</RadioGroup>
```

34. You should now be able to view what the contents of the dialog will look like in the AS design window.
35. We will now return to the `MainActivity.kt` file in order to add the behavior that will allow us to capture click events on the FAB and to accordingly open the dialog to add a new transaction. Add to the class the following, near the other similar variable declarations.

```
private var fab: FloatingActionButton? = null

private var transactionDialog: View? = null
```

36. Within the `onCreate` function, after the block of capturing references to the different UI elements, add the following code. This will get a reference to the FAB, and will specify that on its click, it will execute the `showDialog` function (which we will define next).

```
fab = findViewById(R.id.fab)

fab?.setOnClickListener { view ->
    showDialog()
}
```

37. Within the `MainActivity` class, create the `showDialog` function as below. This function will dynamically create a Material Alert Dialog, and will use our newly created dialog UI within it, in

addition to configuring what the title, message, and action buttons of the dialog should be. Add the imports that are needed within this code block.

```
private fun showDialog() {
    transactionDialog = LayoutInflater.from(this)
        .inflate(R.layout.dialog_transaction, null, false)

    MaterialAlertDialogBuilder(this).setView(transactionDialog)
        .setTitle("Add Transaction")
        .setMessage("Enter transaction details")
        .setPositiveButton("Add") { dialog, _ ->
            dialog.dismiss()
        }
        .setNegativeButton("Cancel") { dialog, _ ->
            dialog.dismiss()
        }
        .show()
}
```

38. Run your application. You should now be able to see the exchange rates as before, and you should also be able to press the FAB button, and have the dialog open and request from you the transaction information. Of course at this stage, if you press 'Add' within the dialog, the dialog will simply close, as per our above code.
39. To provide some organization to our code, remove the code block in `onCreate` that fetches the exchange rates, and move them into a separate function named `fetchRates`, and just call that function from `onCreate`.
40. We will now add the ability to POST a transaction to our backend. We will first create another data model class to represent a transaction. Create a Kotlin class within the `model` package named `Transaction` and add the following contents. As mentioned previously, this will specify the fields of this class, and also specifies what name each field should be given when it is sent to the backend. Note the difference between `usd_amount` which is the name expected by the backend, and `usdAmount` which is the name of the variable used internally within our class.

```
import com.google.gson.annotations.SerializedName

class Transaction() {
    @SerializedName("usd_amount")
    var usdAmount: Float? = null

    @SerializedName("lbp_amount")
    var lbpAmount: Float? = null

    @SerializedName("usd_to_lbp")
    var usdToLbp: Boolean? = null
}
```

41. We now move to the `retrofit.kt` file to add the ability to POST to the transaction endpoint. Within the definition of the `Exchange` interface, add the following signature.

```
@POST("/transaction")
fun addTransaction(@Body transaction: Transaction): Call<Any>
```

42. We now return to `MainActivity` to utilize the functions that will allow posting a transaction to our server. Add the following function within the class. It will accept a transaction object and will post it to the server. It will make use of the Material UI `Snackbar` to provide feedback to the user with the results of the operation.

```
private fun addTransaction(transaction: Transaction) {

ExchangeService.exchangeApi().addTransaction(transaction).enqueue(object :
Callback<Any> {

    override fun onResponse(call: Call<Any>, response:
Response<Any>) {
        Snackbar.make(fab as View, "Transaction added!",
Snackbar.LENGTH_LONG)
            .show()
    }

    override fun onFailure(call: Call<Any>, t: Throwable)
    { Snackbar.make(fab as View, "Could not add transaction.",
Snackbar.LENGTH_LONG)
        .show()
    }

})
}
```

43. Up until this point, we can press the FAB button and open the new transaction dialog. We also have the ability to send a new transaction to the server, and provide the user with appropriate feedback. The only missing piece is to take the user input within the transaction dialog, produce a `Transaction` object from them, and use the `addTransaction` method to send it to the server. We will do this inside the `onClick` listener of the `setPositiveButton` function call.
44. The sample statement below will capture the USD amount of the transaction from the appropriate user input. Add the additional needed code after this statement to capture the LBP amount and the type of the transaction. The following `Radio Group` documentation should be helpful.

```
...
.setMessage("Enter transaction details")
.setPositiveButton("Add") { dialog, _ ->
    val usdAmount =
        transactionDialog?.findViewById<TextInputLayout>
(R.id.txtInptUsdAmount)?.editText?.text.toString()
}
```

```
        .toFloat()

        ...

        dialog.dismiss()
    }
    .setNegativeButton("Cancel") { dialog, _ ->
        ...
    }
```

45. With the above pieces of information, construct an instance of the Transaction class with the corresponding information, and call the `addTransaction` function.
46. Launch your application once again and test that our two key pieces of functionality are in place: viewing the exchange rate, and being able to add a new transaction. Test your application, and confirm that the POST request is being executed successfully by confirming that your transaction is being added to the database as it was entered into the UI, and congratulations! Commit this last phase of changes (as always with an expressive commit message). Tag this code state as `v1` representing the completion of this set of features. Push your commits and tags.